

Text Classification (Sentiment Analysis) using SimpleRNN on IMDb

IMDb Movie Reviews — Binary Sentiment Classification (Negative vs Positive)

Name	Ali Hamza
Roll Number	B23F0063AI106
Section	B.S AI - Red
Course	Artificial Neural Networks Lab (COMP-341L)
Lab Number	08
Date	March 30, 2026
Framework	TensorFlow / Keras

Lab o8 Technical Report

Table of Contents

- Objective
- Task 1 — Dataset
- Task 2 — Dataset requirements & preprocessing
- Task 3 — Model development (SimpleRNN)
- Task 4 — Model training
- Task 5 — Visualization (required graphs)
- Task 6 — Evaluation & sample predictions
- Discussion (5–7 lines)
- Conclusion

Objective

Perform **text classification** (sentiment analysis) using a Recurrent Neural Network built with `SimpleRNN`, trained on a publicly available dataset, and report the complete workflow: dataset details, preprocessing, model development, training curves, evaluation, and sample predictions.

Task 1 — Dataset (Text Classification: Sentiment Analysis)

Requirement	Details (IMDb)
Dataset source	<code>tf.keras.datasets.imdb</code> (IMDb movie reviews) — publicly available dataset distributed with TensorFlow/Keras.
Input format	Each review is a sequence of integers (word IDs), length varies per review.
Output format	Binary label: <code>0</code> = Negative, <code>1</code> = Positive.
Train/Test split	Provided by the dataset loader as separate training and testing sets.

Code description (Task 1): The dataset is loaded with `imdb.load_data(num_words=MAX_FEATURES)` to keep the top frequent words only, then a few decoded samples are shown for qualitative inspection.

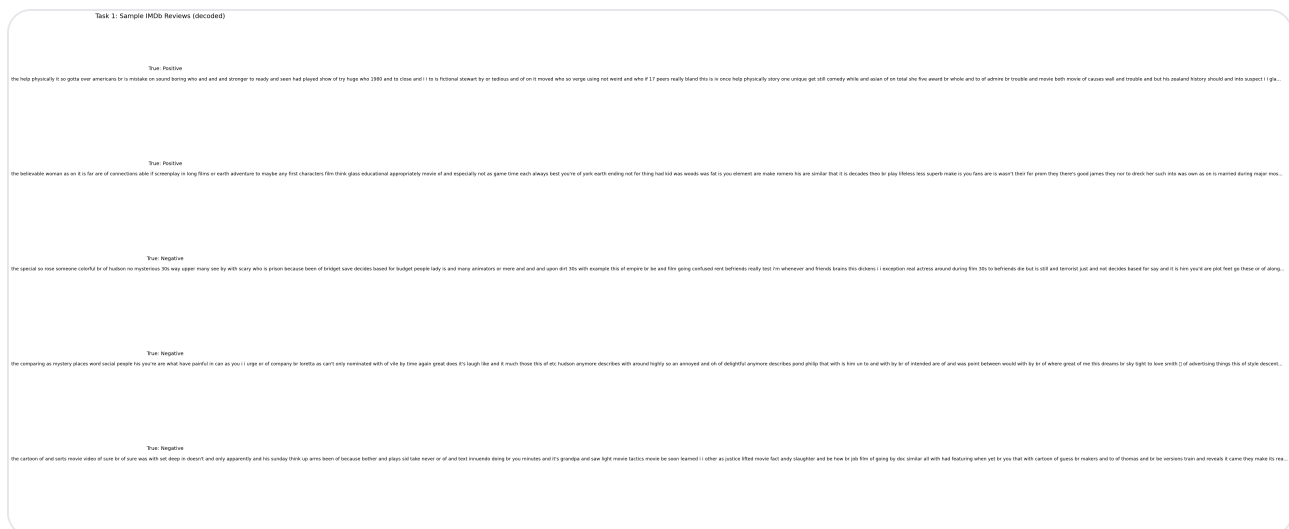


Figure 1. Random decoded review snippets (true labels shown) used to verify dataset loading.

Task 2 — Dataset requirements & preprocessing (Tokenization, Padding, Encoding)

- **Tokenization:** IMDb reviews are already tokenized into integer IDs by Keras.
- **Vocabulary limit:** only the top `MAX_FEATURES=10000` words are kept (rare words are mapped out).

- **Padding & truncation:** reviews are padded/truncated to fixed length `MAX_LEN=200` using `pad_sequences`.
- **Label encoding:** labels remain binary (`0/1`) for `binary_crossentropy`.

Code description (Task 2): The key preprocessing step is sequence padding so the RNN sees a fixed-length batch tensor: `(batch_size, MAX_LEN)`. Post-padding keeps original word order and appends zeros at the end.

```
from tensorflow.keras.preprocessing.sequence import pad_sequences

(x_train_raw, y_train), (x_test_raw, y_test) = tf.keras.datasets.imdb.load_data(num_w
x_train = pad_sequences(x_train_raw, maxlen=200, padding="post", truncating="post")
x_test  = pad_sequences(x_test_raw,  maxlen=200, padding="post", truncating="post")
```

Task 3 — Model development (RNN using SimpleRNN)

The RNN model is built using TensorFlow/Keras with an embedding layer followed by `SimpleRNN` and a sigmoid classifier.

Component	Purpose
<code>Embedding(MAX_FEATURES, EMBEDDING_DIM)</code>	Maps word IDs → dense vectors; learns word representations during training.
<code>SimpleRNN(RNN_UNITS)</code>	Processes the sequence in order and maintains a recurrent hidden state.
<code>Dense(1, sigmoid)</code>	Outputs probability of positive sentiment.

Code description (Task 3): The model is compiled with `adam` optimizer and `binary_crossentropy`, with `accuracy` tracked as the classification metric.

```
model = tf.keras.Sequential([
    tf.keras.layers.Embedding(10000, 64, input_length=200),
    tf.keras.layers.SimpleRNN(64),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

Task 4 — Model training (Loss vs Epochs, Accuracy vs Epochs)

The model is trained on the IMDB training set and a validation subset is used to monitor generalization.

- **Training set:** `x_train`, `y_train` (IMDb training split)
- **Testing set:** `x_test`, `y_test` (IMDb testing split)
- **Validation:** `validation_split=0.2` inside `model.fit`

Recorded (Final Epoch)

Final training accuracy	0.7923
Final validation accuracy	0.5058
Final training loss	0.3210
Final validation loss	1.2359

Code (Training)

```
history = model.fit(
    x_train, y_train,
    batch_size=64, epochs=10,
    validation_split=0.2
)
```

Task 5 — Visualization (Required Graphs)

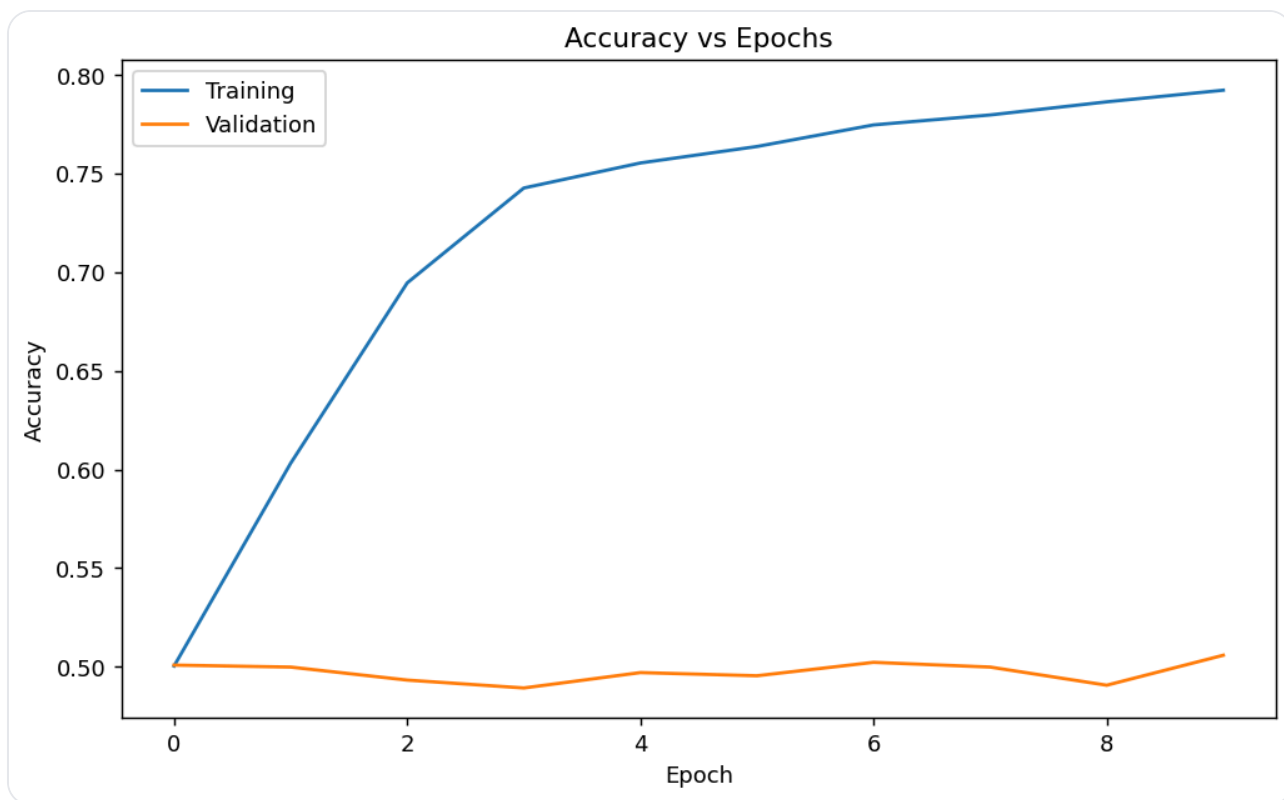


Figure 2. Accuracy vs epochs (training vs validation).

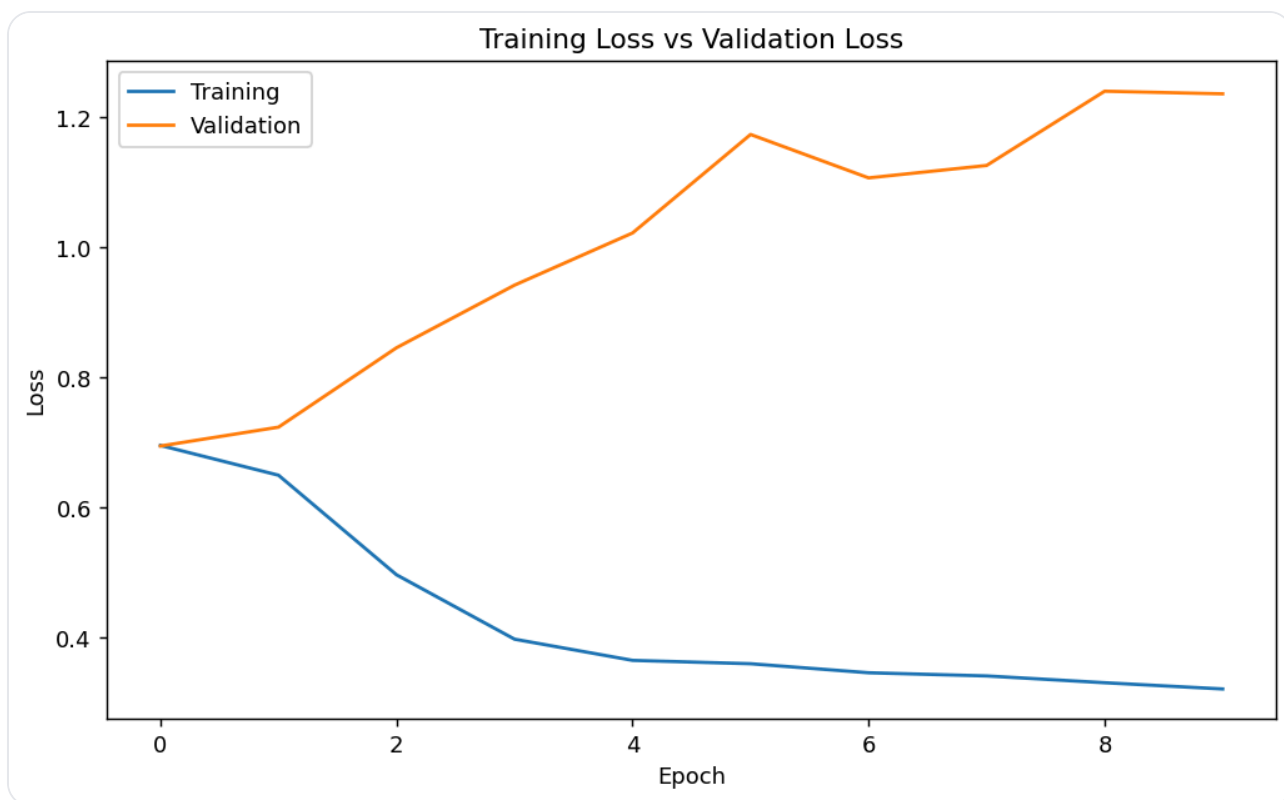


Figure 3. Training loss vs validation loss across epochs.

Task 6 — Evaluation & sample predictions

6.1 Test-set evaluation

Test accuracy	0.5069
Test loss	1.2013

Code description (Evaluation): The model outputs a probability; a threshold of `0.5` converts it into a class. Confusion matrix counts true/false positives/negatives to summarize classification performance.

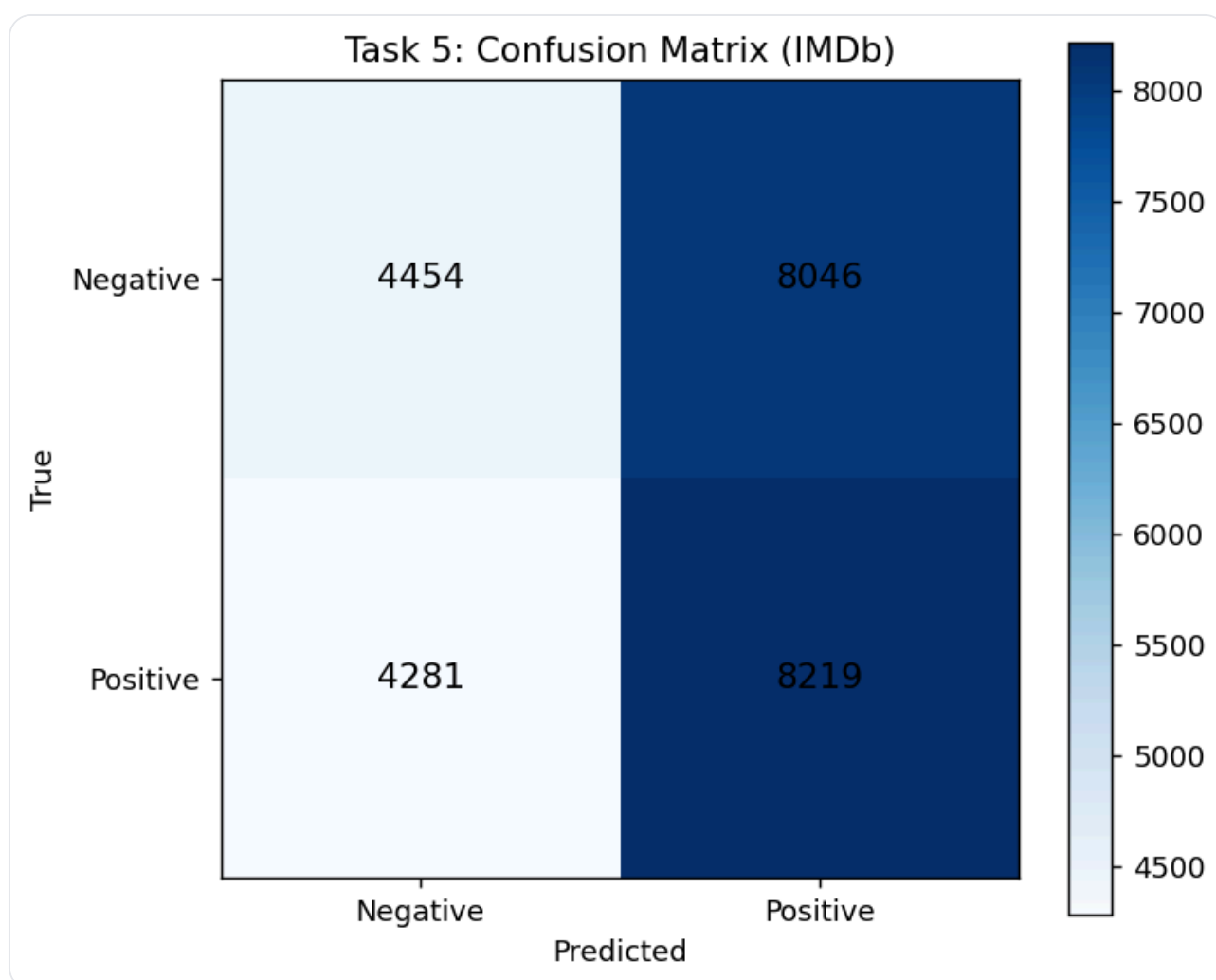


Figure 4. Confusion matrix on the IMDb test set.

6.2 Classification metrics

Sentiment labels: 0=Negative, 1=Positive

Negative: precision=0.5099, recall=0.3563, f1=0.4195

Positive: precision=0.5053, recall=0.6575, f1=0.5715

Confusion Matrix [[TN, FP],[FN, TP]]: [[4454, 8046], [4281, 8219]]

6.3 Sample predictions

Task 6: Sample Predictions (IMDb)	
True: Negative Pred: Negative	the if each in sounds are as on if by to of again characters testament took there his and if each in case she film is everything such to that with characters first is so longer extra proper if today hard reason testament with just be don't great story idea in is overuse teenage kids be and char in movie is everyone joke felt who felt i loved and it out is and then movie friends all and is feeling and or is do late place to this fighting all rate movie who is dislike truly for modest to and they of and hard tryth...
True: Negative Pred: Positive	the when end as on one begins be and to singal instead director find actor are think because happy i doesn't solve critics it who how her do for that it producer of little it slapstick i what have two of passed into at ugly who is do shows film is began caught much as on it but have
True: Negative Pred: Positive	the was although as are of dances to and film just was and she in of what recognize things only sense to test line have does when was after robot read them star too ignorant check cast of couple hard clarify when of clue was after his character that was did as of because blood kind bit was nothing can of by film of apart this that worst when knew it that we it's made her serious such and stephen happen was off just of useful another
True: Positive Pred: Positive	the six pounds clearly of ridiculous bit hundred manage not as my this to have to looked of two black sort of funny unconventional hundred and to adaptations heart of and come alone this and huh but cast he she suit were such changing needed and more and of you gave this psychologically know hundred larger or of berry is so and be is driven deeply away needed tough those this managed act show attention hour to of otherwise in common not not following it price who and unconventional isn't is 3d be eager to good i...
True: Negative Pred: Positive	the was pretty in looking an actually fact many experience some for results while they're be going to could but talk doesn't i misses figures it seems i that it deal for was gang doesn't back to here she better but of hope for my not misses figures with so ahead this as marks was with time better of account for of you spent are by happy unfortunately of you was after his known most was children in can of it is of appears and who of using was after his known doesn't series such was with her length no of you value them...

Figure 5. Five random test samples: decoded text snippet with true vs predicted sentiment.

Discussion (5–7 lines) — write in your own words

- 1) SimpleRNN reads the review *word-by-word* and updates a hidden state (h_t) that summarizes earlier words.
- 2) After the last time step, the final hidden state represents the review and is passed to a sigmoid layer to output a probability of positive sentiment.
- 3) It is suitable here because it is a simple baseline for sequential text where order matters and we want a lightweight recurrent model.
- 4) In my training, the model reached **train acc = 0.7923** but only **val acc = 0.5058**, showing weak generalization.
- 5) On the test set, it achieved **test acc = 0.5069** with **test loss = 1.2013**; the confusion matrix also shows many misclassifications.
- 6) Overall, the curves indicate the model learns the training data but does not transfer well to unseen reviews (likely overfitting / limited capacity of SimpleRNN).

Conclusion — write in your own words

In this lab, I learned how to load a public text dataset (IMDb) where each review is stored as token IDs, and how padding/truncation to a fixed length makes batching possible for RNN training.

I also learned how an Embedding + SimpleRNN pipeline converts discrete word IDs into learned vectors and then processes them sequentially to produce a final sentiment prediction.

From the accuracy/loss curves, I observed a noticeable gap between training and validation performance, which helped me understand generalization and overfitting in practice.

Finally, the confusion matrix and precision/recall values helped me evaluate the model beyond accuracy and see where it is making mistakes.

Assets used: `plots/task1_sample_reviews.png`, `plots/task5_confusion_matrix.png`,
`plots/task6_random_predictions.png`, `plots/task7_accuracy_curve.png`,
`plots/task7_loss_curve.png`.